nl2q: compliant SPARQL from plain language over SPHN clinical graphs

Lorenzo Varese **Michele Inchingolo** **Priyanshi Chandra**
Team Scientific Shrimps
https://github.com/neural-wave/ch-ScientificShrimps

1 Approach

A researcher asks a question in plain language. nl2q returns one SPARQL query over SPHN 2026.1 data in Virtuoso (one named graph per patient), the tokens used, and a verdict on five rules: no single patient data (R1), aggregates over all patients only (R2), no unrestricted match of the whole graph (R3), SELECT only (R4) and no exposed identifiers (R5). Each query runs on 100,000 mock patients with a 10-minute limit, and the evaluation checks the verdict, the time to write and to run the query, and the returned rows. The system must be static: it never queries the database while answering.

Core idea. The *translation layer* writes the query and translates the request *literally*: a request to list the patients with X produces the listing query. The *compliance layer* judges that query with rules written in code (Figure 1). Only syntax and schema errors go back to the generator. A compliance failure sent back would teach the model to turn a forbidden request into an allowed query, and a request that must be refused would come back "compliant". No LLM judges compliance: the verdict comes from a deterministic check of the query's structure, so the same query always gets the same verdict and the wording of a request cannot influence it. The check costs no tokens, takes milliseconds, and its reason names the violated rule.

Processing pipeline (numbered as in Figure 1).

① **Pre-screen (optional).** vllm-sr/d3 judges the request text against 5 labelled requests per rule. It can only block, at confidence ≥ 0.9.

② **Prompt.** System rules; a schema card with one block per SPHN record type (properties, target types, allowed codes): 4 core blocks plus the 6 that best match the request by BM25 over class definitions, and the codes whose labels match; 2 fixed and the 4 most similar of 57 worked examples, built by code from the official queries and unsafe variants of each.

③ **Generator.** Qwen3.8-27B (open weights, Apache-2.0) in FP8 on one GPU, served by vLLM with multi-token-prediction speculative decoding: a recent mid-size model, balancing accuracy and compute.

④ **Finish and check.** SPHN gives every record its own code node, so a query that groups by that node gets one row per record; code replaces the node with its code (e.g. ICD-10-GM E11). Two SPARQL forms Virtuoso rejects are rewritten into accepted ones. Code then adds DEFINE and PREFIX lines. rdflib parses the query and every sphn: term is looked up in SPHN 2026.1; an error goes back, at most twice.

⑤ **Rules.** R1–R5 give the verdict and its reason. The API never runs the query.

Compliance rules. The rules read rdflib's query algebra and track what the query returns. A patient (GRAPH ?g), a record node or an identifier keeps its role through BIND, IF, string functions and every aggregate except COUNT: it may be counted, and returning it, grouping by it or comparing it with a constant is refused (R1, R2, R5). Each group of joined patterns needs a class, code or value (R3); SERVICE is refused (R4). A checker error means non-compliant.

Plan path. We wanted to also provide accurate rows fetched from the query. None of the translator's queries for the 9 compliant official questions returned the reference rows, in any of 24 runs. The same four errors kept appearing: a record type missing from the card, a code node grouped as if it were the code, the root of a value set used as a value, and SQL syntax. Three of the four come from how SPHN stores data. In the plan path, a fine-tuned Qwen3.5-9B writes a short JSON plan of *what* is asked, and a compiler writes *how* to ask it in SPHN. For "How many patients have a billed diagnosis with ICD-10-GM code I10?" the plan is

```
{"records": {"dx": {"type": "BilledDiagnosis",
    "code": ["icd-10-gm:I10"]}},
  "cohort": {"has": "dx"},
  "output": [{"agg": "patients"}]}
```

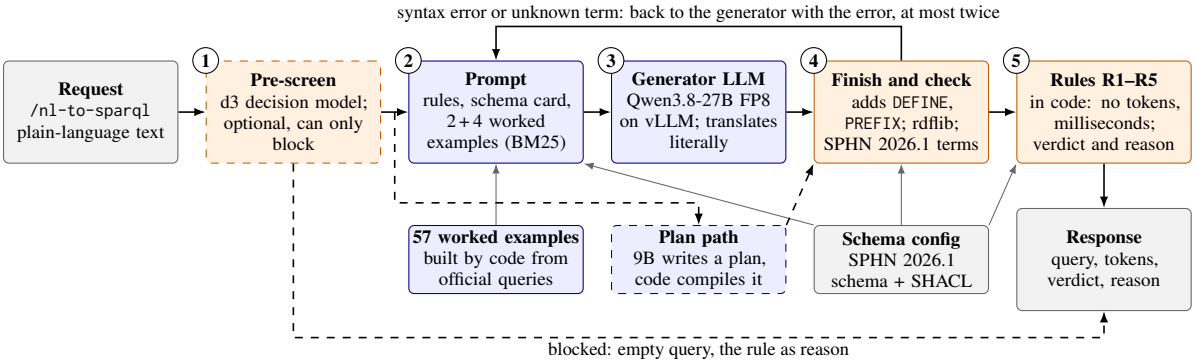


Figure 1: Architecture of `nl2q`. Top: the request path. Bottom: what it reads, built offline. Blue: translation; orange: compliance; grey: shared; dashed: optional. The plan path, when on, replaces stages 2–3.

Besides "has", a plan can say that a patient lacks a record, has one of several, has two records in the same hospital stay, or has two records that differ; it can ask for counts, first or last dates, percentages and the time between two records. Decoding is limited to the plan's JSON schema, and a plan that fails a check (names, record type, values, the question's numbers and codes) goes back to the model once. Requests no plan can express (monthly series, medians) go to free SPARQL, limited by a grammar to SPHN 2026.1 names.

Training data. No labelled SPHN questions exist, so we generate them: code samples a valid plan and writes a template sentence, Qwen3.8-27B rewords it, and a pair is kept if its codes, numbers and units survive. Each pair is correct by construction. Five LoRA adapters (v21–v25) were trained on up to 24,786 pairs, about 20 minutes each on one GPU; v25 adds 5,926 pairs in the style of the official questions. No official question is in the training data. The path is off by default: it needs its own model server.

Query performance. On 612 million triples the order of the triple patterns decides between milliseconds and minutes. As advised by Brites Marto et al. (2025), both the direct generator (stages 2–3) and the plan path's compiler put the code (e.g. E11) before the record type and add `DEFINE sql:select-option "order"`, so Virtuoso keeps that order (two reference queries: 34.6 and 11.3 s without it, 0.10 and 0.04 s with it). The compiler adds rules found by failures at 100,000 patients: each condition is a sub-query listing distinct patients, joined by `UNION` with one flag per patient ("A but not B": 0.29 s; 82 s with `FILTER NOT EXISTS`), and grouping uses plain variables before any expression (an expression gave 70,130 rows where 4 were expected).

Setting	Right	FC	FR	s	Tokens
Rules only, references	26/26	0	0	–	0
Early default	22/26	1	3	3.42	6.8k
Same, reasoning on	25/26	0	1	14.8	8.0k
Submitted default	25/26	1	0	3.4	8.1k
Synthetic set, 3 runs	150/150	0	0	1.8	8.3k

Table 1: Verdicts, on the bench unless stated. FC: false compliant (N10); FR: false refusals; s, tokens: mean per request.

2 Results

Evaluation setup. The *bench* has 26 requests with a label and a reference query, 14 official and 12 ours, 12 compliant; a request never sees its own worked example. With 9 official compliant questions one question is 11 points, so we built a *synthetic set* of 50 items: code compiles 1–3-hop paths of the 2026.1 schema found in the data into gold queries, Apertus v1.5 8B writes each question, and the items come in minimal pairs (aggregate, identifier exposed, identifier used safely) that fix the label. An *easy set* of 30 clear questions (29 scored) was written without access to the plan path. Rows are compared as multisets.

Verdicts (Table 1). The rules alone give the labelled verdict on all 204 labelled queries (48 compliant). The submitted default scores 25/26 with no false refusal, and 150/150 on the synthetic set. N10 asks to count lab results on a remote endpoint; the generator drops the `SERVICE` clause, and the rules judge the query it wrote correctly. Reasoning fixes N10 at about 4 times the latency, so it stays off. Most of the gain over the early default comes from the code-class rewrite: grouping by a code

node caused 29 of its 58 false refusals in 20 runs. d3 blocked 9 of 14 non-compliant requests in every run and no compliant one; it stays off, since a false block cannot be undone. A fuzzer made 154,831 meaning-preserving variants of the labelled queries and found 162 false compliants from two gaps (inverse paths, variables named like rdflib's internal aggregates); after the fix it finds none.

Rows. Free SPARQL returns the reference's rows on 2 of 12 compliant bench requests (0 of 9 official) and on 55% of the synthetic set (numbers 83%, dates 38%, value-set individuals 36%). The plan path is exact on 15 of 29 easy questions, and on 15 of the 18 whose record types its format knows (83%); 11 of its 14 misses need a record type it lacks (drug prescriptions in 6). All 29 queries are valid, with the same 15 exact on 5,000 and on 100,000 patients. Official: 0/9, with O04's count right. With hand-written plans the compiler reproduces the reference numbers on 6 official questions. The model's plans have the right records and conditions and miss details: hours or days, count or percent, outputs.

Execution time at 100,000 patients (611.7 M triples, one query at a time, idle server). The official reference queries for O01 and O04 exceed 10 minutes and O02 takes about 3; the plan path's queries take 0.42 s (hand-written plan), 19 s and 6 s. On the easy set its median is 0.107 s against 0.192 s for the references. The optional optimizer, reading only SHACL 2026.1, speeds up the 80 queries it changes by 1.83× and cuts timeouts at 120 s from 28 to 18.

3 Challenges

Evaluation rules that changed. We shifted our focus away from answer-side privacy; the static rule turned every data-driven idea (k-anonymity, data statistics for the optimizer, a slow path) into a switch, off by default; checking rows made correct rows the target and started the plan path.

Schema versions. The data is SPHN 2024.2: 43 classes of SHACL 2026.1 never occur in it and 15 of its properties are not in 2026.1, so many 2026.1 queries return nothing.

Virtuoso's dialect. 4 of the 9 official queries failed on `ROUND(x,2)` or `EXISTS && EXISTS`; after the rewrite they run, and most still return other rows.

The official question style. Official questions name no code (our training questions: 58%), use vague words (8 of 9; ours 6%) and ask for 3 or more outputs (5 of 6 breakdowns; ours 0.2%).

Ideas that did not work. Better retrieval: card recall from 73.9% to 92.4%, still 0/9. Repair after generation: a semantic verifier gave +1–2 verdicts at +18–25% tokens and the same rows; verifier hints, a path check, a vote over 3–5 drafts and adapting the nearest example stayed at 0/9. No worked examples: 19/26. Plan path: the 9B writing SPARQL wrote `DELETE` or `DESCRIBE`; reasoning used 6× the tokens for fewer valid plans; 5 voted plans agreed on the same wrong plan (0/8); retrying on empty results breaks the static rule.

Known limitations. The rules judge the shape of a query only: a count that matches one patient passes. 1,247 fuzzed variants are falsely refused. The plan format lacks drug prescriptions and returns short code IDs (the references return IRIs).

4 AI agents and LLM usage

No LLM gives the verdict, and no model wrote a worked example or a gold query. We coded with Claude Code (Claude Opus 5.5); experiments ran as Claude Code agents, each in its own git worktree, and we reviewed and merged the results by hand. Qwen3.8-27B rewords the plan path's training sentences and Apertus v1.5 8B writes the synthetic questions, so the model that writes a question never translates it. No terminology label (SNOMED CT, ICD-10-GM, LOINC, CHOP, UCUM) goes into an AI tool. This report was drafted with Claude Code from the repository's notes.

References

Andrea Brites Marto, Philip Krauss, Katie Kalt, Vasundra Touré, Deepak Unni, and Sabine Österle. 2025. Efficient querying of federated large-scale clinical RDF knowledge graphs in the Swiss Personalized Health Network. Preprints.